SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: ***Artificial Intelligence*** **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1 – Breadth-First Search (BFS)

Graph: A is the root; A → B, C; B → D, E; C → F, G. BFS uses a **FIFO queue** and visits nodes level by level, adding children left to right.

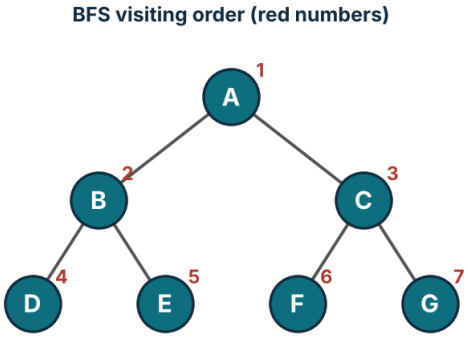


Table: BFS dry run

Iteration	Queue (at start)	Visited Node	Queue (after adding children)
1	A	A	B, C
2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G
5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	(empty)

Answers

1. BFS traversal order: A → B → C → D → E → F → G

2. Queue contents after each iteration: [B, C] → [C, D, E] → [D, E, F, G] → [E, F, G] → [F, G] → [G] → [ ] (empty after iteration 7, so the search ends).

BFS visits every node at depth d (A; then B, C; then D, E, F, G) before any node at depth d + 1, because the queue is first-in-first-out.

Question 2 – Depth-First Search (DFS)

Same graph. DFS uses a **LIFO stack**. Children are pushed right-to-left so that the left child is on top and is explored first. The stack is written with the **top on the left**.

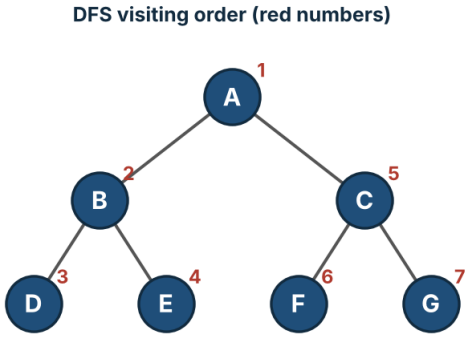


Table: DFS dry run

Iteration	Stack (at start)	Visited Node	Stack (after pushing children)
1	A	A	B, C
2	B, C	B	D, E, C
3	D, E, C	D	E, C
4	E, C	E	C
5	C	C	F, G
6	F, G	F	G
7	G	G	(empty)

DFS traversal order: A → B → D → E → C → F → G

DFS goes as deep as possible along one branch (A → B → D) and only backtracks (to E, then to C) when a leaf is reached.

Question 3 – 8-Puzzle: BFS from Initial to Goal State

Setup: Blank is shown as "_". Moves of the blank are tried in the fixed order **Up, Down, Left, Right**. Duplicate states (already generated) are discarded. States are labelled S0, S1, S2 … in the order they are generated.

Initial State (S0)	Goal State
1 3 6 5 _ 2 4 7 8	1 2 3 4 5 6 7 8 _

Table: first five BFS iterations

Iteration	Current State (expanded)	Queue before expansion	Queue after expansion
1	**S0** 1 3 6 5 _ 2 4 7 8	S0	S1, S2, S3, S4
2	**S1** 1 _ 6 5 3 2 4 7 8	S1, S2, S3, S4	S2, S3, S4, S5, S6
3	**S2** 1 3 6 5 7 2 4 _ 8	S2, S3, S4, S5, S6	S3, S4, S5, S6, S7, S8
4	**S3** 1 3 6 _ 5 2 4 7 8	S3, S4, S5, S6, S7, S8	S4, S5, S6, S7, S8, S9, S10
5	**S4** 1 3 6 5 2 _ 4 7 8	S4, S5, S6, S7, S8, S9, S10	S5, S6, S7, S8, S9, S10, S11, S12

Iteration 1 expands S0 and generates S1 (Up), S2 (Down), S3 (Left), S4 (Right). Iteration 2 expands S1, iteration 3 expands S2, iteration 4 expands S3 and iteration 5 expands S4. BFS continues in this order, level by level; the goal is found at depth 8 and is the **311th** state expanded.

Questions

1. Which node is expanded in each iteration? Iteration 1: S0 (initial state); 2: S1 (blank Up); 3: S2 (blank Down); 4: S3 (blank Left); 5: S4 (blank Right). Nodes are always expanded in the order they were generated (FIFO), i.e. all depth-1 states, then all depth-2 states, and so on, until the goal state (depth 8) is dequeued in iteration 311.

2. How many states are generated in total? Using the move order above and discarding repeated states, BFS generates **500 distinct states** (499 newly generated plus the initial state) and expands 311 of them before the goal is reached. The exact count depends on the move order and duplicate checking.

3. Complete solution path (8 moves, blank moves: Right, Up, Left, Down, Left, Down, Right, Right):

Start	Step 1	Step 2	Step 3	Step 4	Step 5	Step 6	Step 7	Step 8
–	Right	Up	Left	Down	Left	Down	Right	Right
1 3 6 5 _ 2 4 7 8	1 3 6 5 2 _ 4 7 8	1 3 _ 5 2 6 4 7 8	1 _ 3 5 2 6 4 7 8	1 2 3 5 _ 6 4 7 8	1 2 3 _ 5 6 4 7 8	1 2 3 4 5 6 _ 7 8	1 2 3 4 5 6 7 _ 8	1 2 3 4 5 6 7 8 _

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle? Every move has the same cost (1), so a state's path cost equals its depth. BFS expands states in order of increasing depth: all states at depth d are expanded before any state at depth d + 1, because the queue is FIFO. Therefore the first time the goal state appears, it has been reached by a path with the fewest possible moves. BFS is also complete, since the branching factor is finite (at most 4) and this puzzle is solvable (initial and goal states have the same inversion parity). The price is memory: it must store every state on the frontier.